



Tarea 2: Aprendizaje Distribuido para Computer Vision

PROFESOR

Johansell Villalobos Cubillo

2026-06-03

Índice

1 Descripción General	1
2 Objetivos	2
3 Dataset	2
4 Introducción Teórica	3
4.1 Compilación Just-In-Time	4
4.2 Precisión Numérica	4
5 Instrucciones	5
5.1 Configuración del entorno	5
5.2 Implementación del modelo	5
5.3 Entrenamiento	6
6 Experimentos	6
6.1 Impacto del tamaño de lote	6
6.2 Impacto de la tasa de aprendizaje	7
6.3 Impacto del tamaño de la red	7
7 Optimización de Hiperparámetros	7
8 Análisis de Precisión Numérica	8
9 Análisis y Discusión	9
10 Resultados Esperados	9
11 Entrega	10
12 Trabajo Opcional	10

1. Descripción General

Los modelos modernos de Inteligencia Artificial (IA) se entrenan utilizando grandes volúmenes de datos y aceleradores especializados como CPUs multinúcleo, GPUs y TPUs. El crecimiento exponencial de los datos y del tamaño de los modelos ha impulsado el desarrollo de herramientas capaces de aprovechar eficientemente este hardware.

En los últimos años, JAX se ha convertido en uno de los ecosistemas más importantes para investigación y desarrollo en aprendizaje automático debido a su capacidad para combinar programación funcional, diferenciación automática, compilación Just-In-Time (JIT) y ejecución eficiente sobre aceleradores modernos.

Por otra parte, Flax NNX proporciona una interfaz moderna para la construcción de modelos neuronales sobre JAX, facilitando la definición de arquitecturas complejas y la administración del estado del modelo.

En esta tarea se explorará el desarrollo de un sistema de clasificación de imágenes utilizando JAX y Flax NNX. Los estudiantes implementarán, entrenarán y optimizarán una red neuronal convolucional para reconocer imágenes pertenecientes a distintas maravillas del mundo. Además, se analizará el impacto de diferentes hiperparámetros, configuraciones numéricas y técnicas de optimización sobre el rendimiento y la exactitud del modelo.

La actividad está diseñada para ejecutarse sobre una única TPU o GPU disponible en Google Colab, permitiendo a los estudiantes familiarizarse con herramientas modernas utilizadas en sistemas de Inteligencia Artificial de producción e investigación.

2. Objetivos

El propósito de esta tarea es consolidar conceptos relacionados con:

- Programación acelerada utilizando JAX.
- Desarrollo de modelos mediante Flax NNX.
- Diferenciación automática.
- Compilación Just-In-Time (JIT).
- Entrenamiento eficiente sobre GPUs y TPUs.
- Optimización de hiperparámetros.
- Evaluación del impacto de distintas precisiones numéricas.
- Análisis de rendimiento y exactitud.
- Diseño de experimentos reproducibles para aprendizaje profundo.

3. Dataset

Se utilizará el siguiente conjunto de datos:

Wonders of the World Image Classification

Este conjunto de datos contiene imágenes correspondientes a distintas maravillas arquitectónicas y naturales del mundo organizadas en múltiples categorías.

Cada estudiante deberá:

- Descargar el conjunto de datos desde Kaggle.
- Analizar la estructura de clases disponible.
- Realizar el preprocesamiento necesario.
- Redimensionar las imágenes a una resolución adecuada.
- Normalizar los datos de entrada.
- Construir conjuntos de entrenamiento, validación y prueba.
- Implementar un pipeline eficiente para alimentar el acelerador durante el entrenamiento.
- Documentar las decisiones de preprocesamiento adoptadas.

4. Introducción Teórica

El entrenamiento de una red neuronal consiste en encontrar los parámetros que minimizan una función de pérdida asociada al problema de clasificación.

Sea θ el conjunto de parámetros del modelo y $L(\theta)$ la función de pérdida. El descenso por gradiente actualiza los parámetros mediante:

$$\theta_{k+1} = \theta_k - \eta \nabla L(\theta_k) \quad (1)$$

donde:

- θ representa los parámetros del modelo.
- η representa la tasa de aprendizaje.
- ∇L representa el gradiente de la función de pérdida.

Los gradientes son calculados automáticamente mediante el sistema de diferenciación automática de JAX, permitiendo obtener derivadas exactas de funciones complejas de manera eficiente.

4.1. Compilación Just-In-Time

Una de las principales ventajas de JAX es la capacidad de compilar funciones utilizando Just-In-Time Compilation (JIT).

Mediante:

```
1 jax.jit(...)
```

JAX transforma las operaciones a una representación intermedia optimizada que posteriormente es compilada utilizando XLA (Accelerated Linear Algebra).

Esta estrategia permite:

- Reducir tiempos de ejecución.
- Fusionar operaciones matemáticas.
- Optimizar el uso de memoria.
- Aprovechar eficientemente GPUs y TPUs.

Aunque la primera ejecución requiere un tiempo adicional de compilación, las ejecuciones posteriores reutilizan el código optimizado.

4.2. Precisión Numérica

Los aceleradores modernos soportan diferentes representaciones numéricas.

Entre las más utilizadas se encuentran:

- float32
- float16
- bfloat16

La reducción de precisión puede disminuir el consumo de memoria y aumentar el rendimiento computacional. Sin embargo, también puede afectar la estabilidad numérica y la calidad final del entrenamiento.

Por esta razón resulta importante analizar el impacto de cada representación durante el proceso de entrenamiento.

5. Instrucciones

5.1. Configuración del entorno

- Utilice Google Colab.
- Active una GPU o TPU desde:
 - Runtime
 - Change Runtime Type
 - GPU o TPU
- Verifique la disponibilidad de dispositivos utilizando:

```
1 import jax
2
3 print(jax.devices())
4 print(jax.device_count())
```

- Documente el tipo de acelerador utilizado para todos los experimentos.

5.2. Implementación del modelo

- Utilice exclusivamente Flax NNX para implementar el modelo.
- El modelo deberá derivarse de:

```
1 nnx.Module
```

- Utilice componentes apropiados tales como:

```
1 nnx.Conv
2 nnx.Linear
3 nnx.BatchNorm
```

- Se recomienda implementar una arquitectura convolucional.
- Los estudiantes pueden proponer arquitecturas más complejas siempre que justifiquen sus decisiones.

Como referencia mínima, la arquitectura deberá seguir una estructura similar a:

Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Dense → Output (2)

5.3. Entrenamiento

Implemente el ciclo completo de entrenamiento utilizando JAX y Flax NNX.

El entrenamiento deberá utilizar:

- Diferenciación automática.
- Función de pérdida apropiada para clasificación.
- Optimizador basado en gradiente.
- Compilación mediante `jax.jit`.

Reporte:

- Tiempo total de entrenamiento.
- Tiempo promedio por época.
- Exactitud de entrenamiento.
- Exactitud de validación.
- Exactitud de prueba.
- Curvas de pérdida.
- Curvas de exactitud.

6. Experimentos

Realice al menos los siguientes experimentos.

6.1. Impacto del tamaño de lote

Analice al menos tres tamaños de lote:

32, 64, 128

(3)

Reporte:

- Tiempo total de entrenamiento.
- Tiempo promedio por época.
- Exactitud final.
- Throughput.

6.2. Impacto de la tasa de aprendizaje

Evalúe al menos tres tasas de aprendizaje:

$$10^{-2}, \quad 10^{-3}, \quad 10^{-4} \quad (4)$$

Analice la convergencia observada y compare los resultados obtenidos.

6.3. Impacto del tamaño de la red

Evalúe diferentes configuraciones de filtros convolucionales y capas densas.

Por ejemplo:

- 32 filtros
- 64 filtros
- 128 filtros

y

- 128 neuronas
- 256 neuronas
- 512 neuronas

Discuta cómo la complejidad del modelo afecta el desempeño y el tiempo de entrenamiento.

7. Optimización de Hiperparámetros

Realice un estudio sistemático de hiperparámetros.

Como mínimo deberán analizarse:

- Learning Rate.
- Tamaño de lote.
- Número de filtros convolucionales.
- Tamaño de capas densas.
- Número de épocas.

- Tipo de dato utilizado.

Se recomienda utilizar una estrategia organizada de búsqueda, por ejemplo:

- Grid Search.
- Random Search.
- Búsqueda manual guiada por resultados.

Para cada configuración reporte:

- Exactitud final.
- Pérdida final.
- Tiempo total de entrenamiento.
- Throughput.
- Memoria utilizada (si es posible medirla).

Finalmente determine la configuración que proporciona el mejor balance entre:

- Exactitud.
- Tiempo de entrenamiento.
- Consumo de recursos.

8. Análisis de Precisión Numérica

Compare el comportamiento de:

- float32
- float16
- bfloat16

Para cada representación analice:

- Exactitud final.
- Estabilidad numérica.

- Tiempo de entrenamiento.
- Throughput.
- Consumo de memoria.

Discuta las ventajas y desventajas observadas para cada formato.

9. Análisis y Discusión

Responda las siguientes preguntas:

1. ¿Qué ventajas ofrece Flax NNX respecto a implementar modelos directamente en JAX?
2. ¿Qué beneficios aporta la compilación mediante `jax.jit`?
3. ¿Qué hiperparámetro tuvo mayor impacto sobre la exactitud final?
4. ¿Qué hiperparámetro tuvo mayor impacto sobre el tiempo de entrenamiento?
5. ¿Cómo afectó el tamaño de lote al rendimiento observado?
6. ¿Qué diferencias encontró entre float32, float16 y bfloat16?
7. ¿Cuál configuración produjo el mejor balance entre rendimiento y exactitud?
8. ¿Qué limitaciones encontró al utilizar Google Colab?
9. ¿Qué mejoras futuras propondría para aumentar el desempeño del modelo?

10. Resultados Esperados

El informe deberá incluir:

- Descripción completa de la arquitectura utilizada.
- Capturas de pantalla mostrando el acelerador detectado.
- Curvas de pérdida.
- Curvas de exactitud.
- Tablas comparativas de hiperparámetros.

- Tablas comparativas de precisión numérica.
- Gráficas de throughput.
- Discusión de resultados.
- Conclusiones.

11. Entrega

- Notebook de Google Colab completamente funcional.
- Código fuente utilizado.
- Archivo README.md con instrucciones de ejecución.
- Informe en PDF.
- Gráficas y resultados experimentales.

12. Trabajo Opcional

Los estudiantes que dispongan de acceso a múltiples dispositivos podrán realizar un estudio adicional utilizando mecanismos de paralelismo de JAX tales como:

- `jax.pmap`
- Named Sharding
- Positional Sharding
- `pjit`

Analice la escalabilidad obtenida y compare los resultados con la versión de un único dispositivo.